

OOP ↔ SOLID — Where They Meet

Important relationship questions for Junior .NET Developer Interviews

Section	OOP Pillar	SOLID	Qs
Encapsulation & SRP	Encapsulation	S — Single Responsibility	3
Inheritance & LSP	Inheritance	L — Liskov Substitution	4
Polymorphism & OCP	Polymorphism	O — Open/Closed	4
Abstraction & ISP	Abstraction	I — Interface Segregation	4
Encapsulation, Abstraction & DIP	Encapsulation + Abstraction	D — Dependency Inversion	5
Cross-cutting	All OOP Pillars	All SOLID	4

Encapsulation & SRP — protecting what belongs together

Encapsulation + S — Single Responsibility

Encapsulation hides internal state; SRP ensures each class owns exactly one piece of that state and behavior. Together they prevent god-classes and leaking internals.

Q1. How does violating SRP break encapsulation?

Intermediate

Answer: When a class has multiple responsibilities it exposes internal details of each concern to the others, destroying the clean boundary encapsulation should provide. For example a UserService that also sends emails must expose email-related state, leaking implementation details that should be private to an EmailService.

Violates	Follows principles
<pre>class UserService { string _smtpHost; // email detail leaked in void SaveUser(User u) { ... } void SendWelcomeEmail(User u) { ... } }</pre>	<pre>class UserService { private readonly IEmailService _email; void SaveUser(User u) { ... } // only user concern } class EmailService { void Send(string to) { ... } }</pre>

Q2. Why should fields always be private and exposed via properties?

Basic

Answer: Private fields enforce encapsulation — no external code can corrupt state directly. Properties add a controlled gateway: validation logic, change notification, or lazy loading can live in the getter/setter without callers knowing. This also keeps the class closed for modification (OCP) while allowing future extension.

Q3. Can a class follow SRP but still violate encapsulation? Give an example.

Advanced

Answer: Yes. A class may have one responsibility but still expose all its fields as public, allowing external code to manipulate state freely. SRP governs responsibility boundaries; encapsulation governs visibility. Both must be applied. Example: a BankAccount with a single 'manage balance' responsibility still violates encapsulation if Balance is a public field anyone can set.

Inheritance & LSP — safe substitution

Inheritance + L — Liskov Substitution

Inheritance enables code reuse; LSP enforces that derived classes don't betray the contract of their base. An inheritance hierarchy that breaks LSP is a hierarchy that should not exist.

Q4. How does LSP define when inheritance is appropriate?

Intermediate

Answer: Inheritance is appropriate only when the derived class IS a true behavioral subtype of the base — it can replace the base anywhere without surprises. If the derived class must throw `NotSupportedException` or weaken a guarantee the base makes, inheritance is the wrong tool; use composition or a redesigned interface hierarchy instead.

Violates	Follows principles
<pre>class Rectangle { public virtual int Width {get;set;} public virtual int Height {get;set;} } class Square : Rectangle { public override int Width { set { base.Width = base.Height = value; } } // Breaks: rect.Width=4; rect.Height=5; area != 20</pre>	<pre>interface IShape { int Area(); } class Rectangle : IShape { ... } class Square : IShape { ... } // no shared inheritance</pre>

Q5. What is the difference between overriding and violating LSP?

Intermediate

Answer: Overriding is valid when the derived method honours the base contract: same preconditions or weaker, same postconditions or stronger, no new exceptions. Violating LSP means the override breaks the contract — callers relying on base-class behaviour get unexpected results. The keyword `virtual/override` is not enough; semantics must be preserved.

Q6. How can you use unit tests to catch LSP violations?

Advanced

Answer: Write tests against the base class interface and run them against every derived class (the 'Liskov test suite'). If any derived class fails a test that the base passes, LSP is violated. In `xUnit/ NUnit` you can parameterise test classes with a factory delegate that creates each subtype, then run the same assertions on all.

Q7. When should you prefer composition over inheritance to avoid LSP issues?

Advanced

Answer: Prefer composition when: the subtype needs to override many methods with restricted behaviour, when the relationship is 'has-a' rather than 'is-a', or when you need to combine behaviours from multiple sources. Composition avoids the fragile base-class problem and never risks LSP violations because there is no substitutability contract.

Polymorphism & OCP — extending without changing

Polymorphism + O — Open/Closed

Polymorphism is the mechanism; OCP is the goal. When you design with interfaces and virtual dispatch, new behaviour is added by writing new code — not editing old code.

Q8. How does runtime polymorphism enable the Open/Closed Principle?

Intermediate

Answer: By coding to an interface or abstract base, the calling code never needs to change when new types are added. A method that accepts `INotification` and calls `Send()` works for `Email`, `SMS`, or `Push`

without modification — each new channel is a new class, not an edit to existing code.

Violates	Follows principles
<pre>void Notify(string type, string msg) { if (type=="email") SendEmail(msg); else if (type=="sms") SendSms(msg); // must edit for every new type }</pre>	<pre>interface INotification { void Send(string msg); } void Notify(INotification n, string msg) => n.Send(msg); // Add PushNotification : INotification – zero edits here</pre>

Q9. What is the difference between compile-time and runtime polymorphism in the context of OCP?

Intermediate

Answer: Compile-time polymorphism (overloading) adds new method signatures without changing existing ones — a limited form of OCP. Runtime polymorphism (virtual/override, interfaces) is the primary OCP tool: a base reference can be extended with new derived types at any time, and calling code remains untouched.

Q10. How does the Strategy pattern use polymorphism to satisfy OCP?

Advanced

Answer: Strategy defines a family of algorithms behind an interface (e.g. ISortStrategy). The context class holds an ISortStrategy reference and calls Execute(). Adding BubbleSort or QuickSort is adding a new class — the context is never modified. This is OCP applied via composition and runtime polymorphism.

Q11. Can operator overloading violate OCP? Give an example.

Advanced

Answer: Yes, if you add new types to a switch or if-chain inside an overloaded operator. Operator overloading is compile-time polymorphism and cannot dispatch to derived types. If the operator body checks the runtime type to handle new cases, OCP is violated. The fix is to move the logic into virtual methods and let polymorphism dispatch correctly.

Abstraction & ISP — lean contracts

Abstraction + I — Interface Segregation

Abstraction hides implementation behind a contract; ISP keeps that contract minimal. A fat interface exposes too much and forces implementors to deal with concerns they don't own.

Q12. How does a fat interface break abstraction?

Intermediate

Answer: A fat interface forces implementing classes to be aware of concerns outside their domain, polluting their abstraction. A Robot that implements IWorker with Eat() and Sleep() must expose stubs for biological behaviours it has nothing to do with. ISP fixes this by splitting the interface so each implementor only sees what it needs.

Violates	Follows principles
<pre>interface IRepository { void Add(T item); void Delete(T item); void BulkImport(Stream csv); // not all repos need this void ExportToPdf(); // not a repository concern }</pre>	<pre>interface IRepository { void Add(T item); void Delete(T item); } interface IImportable { void BulkImport(Stream csv); } interface IExportable { void ExportToPdf(); }</pre>

Q13. What is the 'role interface' pattern and how does it relate to ISP?

Intermediate

Answer: A role interface is defined from the client's perspective — it contains only the methods the client actually calls. Rather than one IAnimal interface with Move, Eat, Breathe, a locomotion client defines IMovable with Move(). This naturally produces small, focused interfaces that satisfy ISP and improve testability.

Q14. How does ISP improve unit testability?

Intermediate

Answer: Small interfaces are easier to mock. If a service depends on IEmailSender with one method, the test mock is trivial. If it depends on IMessagingService with 15 methods, the mock must stub 14 irrelevant methods. ISP keeps mocks minimal and tests focused on the behaviour under test.

Q15. How do abstract classes and interfaces differ in enforcing abstraction contracts in C#?

Advanced

Answer: An interface is a pure contract with no state — ideal for ISP-style role definitions. An abstract class can carry shared state and partial implementation — useful when several subtypes share common behaviour that should not be duplicated. Choose interface for capability contracts (IDisposable, IComparable); choose abstract class when a base implementation needs sharing alongside the contract.

Encapsulation, Abstraction & DIP — inverting control

Encapsulation + Abstraction + D —
Dependency Inversion

DIP is OOP principles applied to dependency design: hide concrete implementations behind abstractions (encapsulation) and code to interfaces (abstraction). The result is a system where high-level policy is never coupled to low-level detail.

Q16. How do encapsulation and DIP work together to decouple modules?

Intermediate

Answer: Encapsulation hides the internal implementation of a class. DIP requires that the class is accessed only through its abstraction (interface). Together: callers never see the concrete type, so the implementation can change freely. This is the foundation of the Repository pattern — callers see only IOrderRepository and are unaware whether SQL, MongoDB, or an in-memory store is used.

Violates	Follows principles
<pre>class ReportService { // Directly depends on concrete class private SqlReportRepository _repo = new SqlReportRepository(); }</pre>	<pre>class ReportService { private readonly IReportRepository _repo; public ReportService(IReportRepository repo) { _repo = repo; } // Works with SQL, Mongo, in-memory – caller decides }</pre>

Q17. What is the difference between Dependency Injection and the Dependency Inversion Principle?

Intermediate

Answer: DIP is a design principle — high-level modules depend on abstractions. Dependency Injection (DI) is a technique that satisfies DIP by providing (injecting) the concrete dependency from outside the class. DI is one implementation of DIP. You can violate DIP even with DI if you inject a concrete class rather than an interface.

Q18. How does the Factory pattern relate to DIP?**Advanced**

Answer: A Factory creates concrete objects but exposes them through an interface, so the consumer only depends on the abstraction. This satisfies DIP: the high-level class calls `IShapeFactory.Create()` and receives an `IShape` — it never references `Circle` or `Square` directly. The factory encapsulates the 'which concrete type' decision.

Q19. What are DI lifetime pitfalls in ASP.NET Core that violate DIP intent?**Advanced**

Answer: Registering a concrete class directly (`services.AddScoped()`) couples callers to the implementation even through the DI container. Always register as `services.AddScoped()`. The captive dependency problem (Singleton holding Scoped) also violates DIP intent: the singleton is now indirectly coupled to request-lifecycle details.

Q20. How would you apply DIP when integrating a third-party library such as a payment SDK?**Advanced**

Answer: Wrap the third-party SDK behind your own interface (`IPaymentGateway`). Create an adapter class (`StripePaymentGateway : IPaymentGateway`) that calls the SDK. Register the adapter in DI. Your application code depends only on `IPaymentGateway` and is completely isolated from the SDK. Swapping to a different provider is one line in `Program.cs`.

Cross-cutting — all principles together*All OOP Pillars + All SOLID*

The most valuable interview questions connect multiple principles. Mastering these shows you understand the relationships, not just the definitions.

Q21. How do the four pillars of OOP map to the SOLID principles?**Intermediate**

Answer: Encapsulation -> SRP (one responsibility, hidden implementation) and DIP (hide concrete behind interface). Inheritance -> LSP (only inherit when substitutability holds) and OCP (extend via subclass without modifying base). Polymorphism -> OCP (new types extend behaviour) and DIP (code to abstractions dispatched polymorphically). Abstraction -> ISP (lean interfaces) and DIP (depend on the abstraction layer).

Q22. A colleague says 'just make everything an interface and inject it — that satisfies all SOLID'. What is wrong with that?**Advanced**

Answer: Blindly interfacing everything violates ISP (fat interfaces), can break SRP (interfaces that do too much), and adds accidental complexity. SOLID is about intent: interfaces should model real roles (ISP), classes should own one concern (SRP), hierarchies must honour contracts (LSP), extension points should be deliberate (OCP), and dependencies should flow toward stable abstractions (DIP). Mechanics without intent produces over-engineered, hard-to-read code.

Q23. How would you refactor a God Class to satisfy SRP, OCP, and DIP simultaneously?**Advanced**

Answer: 1. Identify responsibilities in the god class (SRP). 2. Extract each into its own class. 3. Define an interface per extracted class (DIP + ISP). 4. Wire them via constructor injection. 5. Ensure the original caller now depends only on abstractions (DIP). Future new behaviour is a new class implementing an existing interface (OCP) — no existing class needs to be edited.

Q24. Why is the Decorator pattern a perfect example of OCP, SRP, and LSP working together?**Advanced**

Answer: OCP: new behaviour is added by wrapping, not by modifying. SRP: each decorator has one job (logging, caching, validation). LSP: the decorator implements the same interface as the wrapped object, so it is fully substitutable. The pattern demonstrates all three in a single structure and is a strong interview answer for 'show me SOLID in action'.

Interview tip: connect OOP concepts to SOLID in every answer — it shows architectural thinking, not just syntax knowledge.